

EX:

```
1) class A {  
2)     int i=10;  
3)     int j=20;  
4)     A(int i,int j) {  
5)         System.out.println(i+" "+j);  
6)         System.out.println(this.i+" "+this.j);  
7)     }  
8) }  
9) class Test {  
10)     public static void main(String args[]) {  
11)         A a = new A(30,40);  
12)     }  
13) }
```

OUTPUT:

30 40
10 20

NOTE: In general, in enterprise applications, we are able to write no. of Java bean classes as per application requirement. In Java bean classes, we are able to provide variables and their setter methods and getter methods. In Java bean classes, in setter methods, we have to declare parameter variable with the same name of the respective class level variable, here we have to transfer data from local variable to class level variable, for this, we have to assign local variable to class level variable. In this context, to refer class level variable over local variable separately we have to use 'this' keyword. In getter methods, always, we have to return class level variables only, so that, it is not required to use 'this' keyword.

EX:

```
1) class User {  
2)     private String uname;  
3)     private String upwd;  
4)     public void setUsername(String uname) {  
5)         this.uname = uname;  
6)     }  
7)     public void setUpwd(String upwd) {  
8)         this.upwd = upwd;  
9)     }  
10)    public getUsername() {  
11)        return uname;  
12)    }  
13)    public getUpwd() {  
14)    }  
15) }
```

```

16) class Test {
17)     public static void main(String[] args) {
18)         User u = new User();
19)         u.setUname("abc");
20)         u.setUpwd("abc123");
21)         System.out.println("User Login Details");
22)         System.out.println("-----");
23)         System.out.println("User Name :"+u.getUserName());
24)         System.out.println("Password :"+getEaddr());
25)     }
26) }

```

EX:

```

1) class Employee
2) {
3)     private String eid;
4)     private String ename;
5)     private float esal;
6)     private String eaddr;
7)
8)     public void setEid(String eid)
9)     {
10)         this.eid=eid;
11)     }
12)     public void setName(String ename)
13)     {
14)         this.ename=ename;
15)     }
16)     public void setEsal(float esal)
17)     {
18)         this.esal=esal;
19)     }
20)     public void setEaddr(String eaddr)
21)     {
22)         this.eaddr=eaddr;
23)     }
24)
25)     public String getEid()
26)     {
27)         return eid;
28)     }
29)     public String getName()
30)     {
31)         return ename;
32)     }

```

```

33) public float getEsal()
34) {
35)     return esal;
36) }
37) public String getEaddr()
38) {
39)     return eaddr;
40) }
41) }
42) class Test
43) {
44)     public static void main(String[] args)
45)     {
46)         Employee emp=new Employee();
47)         emp.setEid("E-111");
48)         emp.setName("AAA");
49)         emp.setEsal(5000.0f);
50)         emp.setEaddr("Hyd");
51)
52)         System.out.println("Employee Details");
53)         System.out.println("-----");
54)         System.out.println("Employee Id   :"+emp.getId());
55)         System.out.println("Employee Name  :"+emp.getName());
56)         System.out.println("Employee Salary :"+emp.getEsal());
57)         System.out.println("Employee Address :"+emp.getEaddr());
58)     }
59) }

```

2) To Refer Current Class Method:

If we want to refer current class method by using 'this' keyword then we have to use the following syntax.

```
this.method_Name([param_List]);
```

Note: To access current class methods, it is not required to use any reference variable and any keyword including 'this', directly we can access.

EX:

```

1) class A {
2)     void m1() {
3)         System.out.println("m1-A");
4)     }
5)     void m2() {
6)         System.out.println("m2-A");

```

```

7)     m1();
8)     this.m1();
9) }
10)}
11) class Test {
12)     public static void main(String args[]) {
13)         A a = new A();
14)         a.m2();
15)     }
16)}

```

OUTPUT:

m2-A

m1-A

m1-A

3) To Refer Current Class Constructors:

If we want to refer current class constructor by using 'this' keyword then we have to use the following syntax.

this([param_List]);

EX:

```

1) class A {
2)     A() {
3)         this(10);
4)         System.out.println("A-0-arg-con");
5)     }
6)     A(int i) {
7)         this(22.22f);
8)         System.out.println("A-int-param-con");
9)     }
10)    A(float f) {
11)        this(33.3333);
12)        System.out.println("A-float-param-con");
13)    }
14)    A(double d) {
15)        System.out.println("A-double-param-con");
16)    }
17)}
18) class Test {
19)     public static void main(String args[]) {
20)         A a = new A();
21)     }

```

```
|22) }
```

OUTPUT:

A-double-param-con

A-float-param-con

A-int-param-con

A-0-arg-con

NOTE: In the above program we have provided more than one constructor with the same name and with the different parameter list, this process is called as "Constructor Overloading".

In the above program, we have called all the current class constructors by using 'this' keyword in chain fashion, this process is called as "Constructor Chaining".

NOTE: If we want to access current class constructor by using 'this' keyword then the respective 'this' statement must be provided as first statement, if we have not provided 'this' statement as first statement then compiler will rise an error like 'call to this must be first statement in constructor'.

EX:

```
1) class A {
2)     A() {
3)         System.out.println("A-0-arg-con");
4)         this(10);
5)     }
6)     A(int i) {
7)         System.out.println("A-int-param-con");
8)         this(22.22f);
9)     }
10)    A(float f) {
11)        System.out.println("A-float-param-con");
12)        this(33.3333);
13)    }
14)    A(double d) {
15)        System.out.println("A-double-param-con");
16)    }
17) }
18) class Test {
19)     public static void main(String args[]) {
20)         A a = new A();
21)     }
22) }
```

Status: Compilation Error

NOTE: If we want to refer current class constructor by using 'this' keyword then the respective 'this' statement must be provided in the current class another constructor only, not in normal Java methods. If we violate this condition then compiler will rise an error like 'call to this must be first statement in constructors'.

EX:

```
1) class A
2) {
3)     A()
4)     {
5)         System.out.println("A-0-arg-con");
6)     }
7)     A(int i)
8)     {
9)         System.out.println("A-int-param-con");
10)    }
11)    void m1()
12)    {
13)        this(10);
14)        System.out.println("m1-A");
15)    }
16) }
17) class Test
18) {
19)     public static void main(String args[])
20)     {
21)         A a=new A();
22)         a.m1();
23)     }
24) }
```

Status: Compilation Error.

Q) Is it possible to refer more than one current class constructors by using 'this' keyword from a single current class constructor?

No, It is not possible to refer more than one current class constructors by using 'this' keyword, because, in constructors, 'this' statement must be first statement while referring current class constructors. If we provide more than one time this(--) statement then only one this() statement is first statement among multiple this statements, in this case, compiler will rise an error like 'call to this must be first statement'.

EX:

```
1) class A
2) {
3)     A()
4)     {
5)         this(10);
6)         this(22.22f);
7)         System.out.println("A-0-arg-con");
8)     }
9)     A(int i)
10)    {
11)        System.out.println("A-int-param-con");
12)    }
13)    A(float f)
14)    {
15)        System.out.println("A-float-param-con");
16)    }
17) }
18) class Test
19) {
20)     public static void main(String args[])
21)     {
22)         A a=new A();
23)     }
24) }
```

Status: Compilation Error

4) To Return Current Class Object:

To return current class object by using 'this' keyword then we have to use the following syntax.

return this;

EX:

```
1) class A {
2)     A getRef1() {
3)         A a = new A();
4)         return a;
5)     }
6)     A getRef2() {
7)         return this;
8)     }
```

```

9)  }
10) class Test {
11)     public static void main(String args[]) {
12)         A a = new A();//[a=abc123]
13)         System.out.println(a);
14)         System.out.println();
15)         System.out.println(a.getRef1());//[abc123.getRef1()]
16)         System.out.println(a.getRef1());//[abc123.getRef1()]
17)         System.out.println(a.getRef1());//[abc123.getRef1()]
18)         System.out.println();
19)         System.out.println(a.getRef2());//[abc123.getRef2()]
20)         System.out.println(a.getRef2());//[abc123.getRef2()]
21)         System.out.println(a.getRef2());//[abc123.getRef2()]
22)     }
23) }

```

OUTPUT:

A@5e3a78ad

A@50c8d62f

A@3165d118

A@138297fe

A@5e3a78ad

A@5e3a78ad

A@5e3a78ad

In the above program, for every call of getRef1() method JVM will encounter "new" keyword, JVM will create new Object for class A every time and JVM will return new object reference value every time. This approach will increase no. of objects in Java application, it will not provide Objects reusability, it will provide object duplication, it is not suggestible in application development.

In the above program, for every call of getRef2() method JVM will encounter "return this" statement, JVM will not create new Object for class A every time, JVM will return the same reference value on which we have called getRef2() method. This approach will increase Objects reusability.

'static' keyword:

'static' is a Java keyword, it will improve sharability in Java applications.

In Java applications, static keyword will be utilized in the following four ways.

- 1) Static variables
- 2) Static methods
- 3) Static blocks
- 4) Static import

1) Static variables:

- Static variables are normal Java variables, which will be recognized and executed exactly at the time of loading the respective class byte code to the memory.
- Static variables are normal java variables, they will share their last modified values to the future objects and to the past objects of the respective class.
- In Java applications, static variables will be accessed either by using the respective class reference variable or by using the respective class name directly.

NOTE: To access static variables we can use the respective class reference variable which may or may not have reference value. To access static variables it is sufficient to take a reference variable with null value.

NOTE: If we access any non-static variable by using a reference variable with null value then JVM will rise an exception like "java.lang.NullPointerException". If we access static variable by using a reference variable contains null value then JVM will not rise any exception.

In Java applications, static variables must be declared as class level variables only, they never be declared as local variables.

In Java applications, static variable values will be stored in method area, not in Stack memory and not in Heap Memory.

In Java applications, to access current class static variables we can use "this" keyword.

EX:

```
1) class A {
2)     static int i=10;
3)     int j = 10;
4)     void m1() {
5)         //static int k=30;--->error
6)         System.out.println("m1-A");
7)         System.out.println(this.i);
8)     }
9) }
10) class Test {
11)     public static void main(String args[]) {
12)         A a = new A();
13)         System.out.println(a.i);
14)         System.out.println(A.i);
15)         a.m1();
```

```

16)         A a1 = null;
17)         //System.out.println(a1.j);--->NullPointerException
18)         System.out.println(a1.i);
19)     }
20) }

```

OUTPUT:

```

10
10
m1-A
10
10

```

EX:

```

1) class A {
2)     static int i=10;
3)     int j=10;
4) }
5) class Test {
6)     public static void main(String args[]) {
7)         A a1 = new A();
8)         System.out.println(a1.i+" "+a1.j);
9)         a1.i=a1.i+1;
10)        a1.j=a1.j+1;
11)        System.out.println(a1.i+" "+a1.j);
12)        A a2 = new A();
13)        System.out.println(a2.i+" "+a2.j);
14)        a2.i=a2.i+1;
15)        a2.j=a2.j+1;
16)        System.out.println(a1.i+" "+a1.j);
17)        System.out.println(a2.i+" "+a2.j);
18)        A a3 = new A();
19)        System.out.println(a3.i+" "+a3.j);
20)        a3.i=a3.i+1;
21)        a3.j = a3.j+1;
22)        System.out.println(a1.i+" "+a1.j);
23)        System.out.println(a2.i+" "+a2.j);
24)        System.out.println(a3.i+" "+a3.j);
25)    }
26) }

```

OUTPUT:

```

10 10
11 11
11 10

```

12 11
12 11
12 11
12 10
13 11
13 11
13 11

NOTE: In Java applications, Instance variable is specific to each and every object that is a separate copy of instance variables will be maintained by each and every object but static variable is common to every object that is the same copy of static variable will be maintained by all the objects of the respective class.

Note: In Java, for a particular, class Byte-Code will be loaded only one time but we can create any number of objects.

2) **Static Methods:**

- Static method is a normal java method, it will be recognized and executed the time when we access that method.
- Static methods will be accessed either by using reference variable or by using the respective class name directly.

Note: In the case of accessing static methods by using reference variables, reference variable may or may not have object reference value, it is possible to access static methods with the reference variables having 'null' value. If we access non-static method by using a reference variable contains null value then JVM will rise an exception like "java.lang.NullPointerException".

Static methods will allow only static members of the current class, static methods will not allow non-static members of the current class directly.

Note: If we want to access non-static members of the current class in static methods then we have to create an object for the current class and we have to use the generated reference variable.

Static methods are not allowing 'this' keyword in its body but to access current class static methods we are able to use 'this' keyword.

EX:

```
1) class A {  
2)     int l = 10;  
3)     static int j = 20;  
4)     static void m1() {  
5)         System.out.println("m1-A");  
6)         System.out.println(i);  
}
```

```

7)      //System.out.println(i);---->error
8)      //System.out.println(this.j);----->error
9)      A a = new A();
10)     System.out.println(a.i);
11)     }
12)     void m2() {
13)         System.out.println("m2-A");
14)         this.m1();
15)     }
16) }
17) class Test {
18)     public static void main(String[] args) {
19)         A a = new A();
20)         a.m1();
21)         a = null;
22)         a.m1();
23)         A.m1();
24)     }
25) }

```

OUTPUT:

```

m1-A
20
10
m1-A
20
10
m1-A
20
10

```

Q) Is it possible to print a line of text on command prompt without using main() method?

Yes, it is possible to display a line of text on command prompt without using main() method, but, by using static variable and static method combination.

EX:

```

1) class Test {
2)     static int i = m1();
3)     static int m1() {
4)         System.out.println("Welcome to durga software souldtions");
5)         System.exit(0); //to terminate the application
6)         return 10;
7)     }

```

8) }

OUTPUT: Welcome to durga software soultions

If we provide 'Test' class name along with 'java' command on command prompt then JVM will take main class from command prompt, JVM will search for its .class file, if it is available then JVM will load main class bytecode to the memory that is Test class bytecode. At the time of loading Test class bytecode to the memory, JVM will recognize and initialize static variable, as part of initialization, JVM will execute static method. By the execution of static method, JVM will display the required message on command prompt, when JVM encounter System.exit(0) statement then JVM will terminate the application.

NOTE: The above question and answer are valid up to JAVA6 version, it is invalid from JAVA7 version, because, In JAVA6 version JVM will load main class bytecode to the memory irrespective of main() method availability. In JAVA7, first, JVM will check whether main() method is existed or not in main class, if main() method is available then only JVM will load main class bytecode to the memory, if main() method is not available in main class then JVM will not load main class bytecode to the memory and JVM will provide the following error message.

Error:Main Method not found in class Test,please define the main method as:
public static void main(String args[])

3) **Static Block:**

- Static Block is a set of instructions, which will be recognized and executed at the time of loading the respective class bytecode to the memory.
- Static blocks are able to allow static members of the current class directly, Static blocks are not allowing non-static members of the current class directly.

Note: If we want to access non-static members of the current class in static block then we must create object for the respective class and we have to use the generated reference variable.

Static blocks are not allowing 'this' keyword in its body.

EX:

```
1) class A {  
2)     int i=10;  
3)     static int j=20;
```

```

4) static {
5)     System.out.println("SB-A");
6)     System.out.println(i); //----->Error
7)     A a = new A();
8)     System.out.println(a.i);
9)     System.out.println(j);
10)    System.out.println(this.j);----->Error
11) }
12) }
13) class Test {
14)     public static void main(String[] args) {
15)         A a = new A();
16)     }
17) }

```

OUTPUT:

SB-A

10

20

Q) Is it Possible to print a line of text on command prompt with out using main() method, static variable and static method?

Yes, it is possible to display a line of text on command prompt without using main() method, static variable, static method but by using static block.

EX:

```

1) class Test {
2)     static {
3)         System.out.println("Welcome to DurgaSoftware Souldtions");
4)         System.exit(0); //To terminate the programme
5)     }
6) }

```

OUTPUT:

Welcome to DurgaSoftware Souldtions

If we provide main class name 'Test' along with 'java' command on command prompt then JVM will take main class name i.e Test and JVM will search for its .class file. If Test.class file is identified then JVM will load its bytecode to the memory, at the time of loading main class bytecode to the memory static block will be executed, with this, the required message will be displayed on command prompt. When JVM encounter System.exit(0) then JVM will terminate the program execution.

NOTE: The above question and answer are valid up to JAVA6 version, they are invalid from JAVA7 version onwards, because, in JAVA6 version JVM will load main class bytecode to the memory without checking main() method availability but in JAVA 7, first, JVM will search for main() method, if it is available then only JVM will load main class bytecode to the memory, if it is not existed then JVM will provide the following Error message.

Error: Main method not found in class Test, please define the main method as:
`public static void main(String args[])`

Q)Is it possible to display a line of text on command prompt without using main() method, static variable, static method, static block?

Yes, it is possible to display a line of text on command prompt without using main() method, a static variable, a static method and static block but by using "static Anonymous Inner class of Object class".

EX:

```
1) class Test {  
2)     static Object obj = new Object() {  
3)         {  
4)             System.out.println("Welcome To durga Software Solutions");  
5)             System.exit(0); //TO terminate the application.  
6)         }  
7)     };  
8) }
```

OUTPUT:

JAVA6:Welcome to Durga Software Soutions

JAVA7>Error:main method not found in class Test,please define the main method as:
`public static void main(String[] args)`

4) static import:

- In Java, applications, if we import a particular package to the present java file then it is possible to access all the classes and interfaces of that package directly without using package name every time as fully qualified name.
- If we want to access classes and interfaces of a particular package without importing then we must use fully qualified name every time that is we have to use package name along with class names.
- A java program without import statement:

```
java.io.BufferedReader br=new java.io.BufferedReader(new  
java.io.InputStreamReader(System.in));
```

- A java program With import statement:
`import java.io.*;`
`BufferedReader br = new BufferedReader(new InputStreamReader(System.in));`
- In Java applications, if we want to access static members of a particular class in present java file then we must use either reference variable of the respective class or directly class name.
- In Java applications, if we want to access static members without using the respective class name and without using the respective class reference variable then we have to import static members of that respective class in the present Java file.
- To import static members of a particular class in the present java file, JDK 5.0 version has provided a new feature called as "static import".

Syntaxes:

- 1) `import static package_Name.Class_Name_Or_Interface_Name.*;`
It will import all the static members from the specified class or interface.
- 2) `import static package_Name.Class_Name_Or_Interface_Name.member_Name;`
It will import only the specified member from the specified class or interface.

EX:

```
1) import static java.lang.Thread.*;  
2) import static java.lang.System.out;  
3) class Test {  
4)     public static void main(String args[]) {  
5)         out.println(MIN_PRIORITY);  
6)         out.println(MAX_PRIORITY);  
7)         out.println(NORM_PRIORITY);  
8)     }  
9) }
```

OUTPUT:

```
1  
10  
5
```


Static Context / Static Flow of Execution:

In Java, Static Context will be represented in the form of the following 3 elements.

- 1) Static variables.
- 2) Static methods.
- 3) Static blocks

In Java applications, instance context will be created separately for each and every object but static context will be created for each and every class.

In Java applications, static context will be recognized and executed exactly at the time of loading the respective class bytecode to the memory.

In Java applications, when we create object for a particular class, first, JVM has to access constructor, before executing constructor, JVM has to load the respective class bytecode to the memory.

At the time of loading class bytecode to the memory, JVM has to recognize and execute static context.

EX:

```
1) class A {  
2)     static {  
3)         System.out.println("SB-A");  
4)     }  
5)     static int i = m1();  
6)     static int m1() {  
7)         System.out.println("m1-A");  
8)         return 10;  
9)     }  
10) }  
11) class Test {  
12)     public static void main(String args[]) {  
13)         A a = new A();  
14)     }  
15) }
```

OUTPUT:

SB-A

m1-A

EX:

```
1) class A {  
2)     static int i = m2();  
3)     static int m1() {  
4)         System.out.println("m1-A");
```

```

5)         return 10;
6)     }
7)     static {
8)         System.out.println("SB-A");
9)     }
10)    static int m2() {
11)        System.out.println("m2-A");
12)        return 20;
13)    }
14)    static int j=m1();
15) }
16) class Test {
17)     public static void main(String args[]) {
18)         A a1 = new A();
19)         A a2 = new A();
20)     }
21) }

```

OUTPUT:

m2-A

SB-A

m1-A

EX:

```

1) class A {
2)     static int i = m2();
3)     A() {
4)         System.out.println("A-con");
5)     }
6)     int m1() {
7)         System.out.println("m1-A");
8)         return 10;
9)     }
10)    static {
11)        System.out.println("SB-A");
12)    }
13)    int j = m1(); {
14)        System.out.println("IB-A");
15)    }

```

```

16) static int m2() {
17)     System.out.println("m2-A");
18)     return 10;
19) }
20) }
21) class Test {
22)     public static void main(String args[]) {
23)         A a1 = new A();
24)         A a2 = new A();
25)     }
26) }

```

OUTPUT:

```

m2-A
SB-A
m1-A
IB-A
A-con
m1-A
IB-A
A-con

```

Factory Method:

Factory Method is a method, it can be used to return the same class Object where we have declared that method.

Factory Method is an idea provided by a design pattern called as "Factory Method Design Pattern".

NOTE: Design pattern is a System, it will provide problem definition and its solution in order to solve design problems.

EX:

```

1) class A {
2)     private A() {
3)         System.out.println("A-con");
4)     }
5)     void m1() {
6)         Svstem.out.println("m1-A");

```

```

7)    }
8)    static A getRef()//Factory Method
9)    {
10)   A a = new A();
11)   return a;
12)   }
13) }
14) class Test {
15)   public static void main(String args[]) {
16)       A a = A.getRef();
17)       a.m1();
18)   }
19) }

```

OUTPUT:

A-con
m1-A

There are 2 types of Factory Methods

- 1) Static Factory Method
- 2) Instance Factory Method

Static Factory Method:

Static Factory Method is a static method returns the same class object.

EX:

```

Class c = Class.forName(--);
NumberFormat nf = NumberFormat.getInstance(-);
DateFormat df = DateForamt.getInstance(--);
ResourceBundle rb = ResourceBundle.getBundle(--);

```

Instance Factory Method:

If any non-static method returns the same class object then that method is called as "Instance Factory Method".

EX: Allmost all the String class methods are Instance Factory methods.

```

String str = new String("DurgaSoftware Solutions");
String str1 = str.concat(" Hyderabad");
String str2 = str.trim();

```

```
String str3 = str.toUpperCase();  
String str4 = str.substring(5,14);
```

Singleton Class:

If any JAVA class allows to create only one Object then that class is called as "Singleton Class".

Singleton Class is an idea provided by a design pattern called as "Singleton Design Pattern".

EX1:

```
1) class A {  
2)     static A a = null;  
3)     private A() {  
4)     }  
5)     static A getRef() {  
6)         if(a == null) {  
7)             a = new A();  
8)             return a;  
9)         }  
10)        else {  
11)            return a;  
12)        }  
13)    }  
14)}  
15) class Test {  
16)     public static void main(String args[]) {  
17)         System.out.println(A.getRef());  
18)         System.out.println(A.getRef());  
19)         System.out.println(A.getRef());  
20)     }  
21)}
```

OUTPUT: A@a6eb38a
A@a6eb38a
A@a6eb38a

Another Alternative for Singleton Class:

```
1) class A {  
2)     static A a = null;  
3)     static {  
4)         a = new A();  
5)     }  
6)     private A() {
```

```

7)    }
8)    static A getRef() {
9)        return a;
10)   }
11) }
12) class Test {
13)     public static void main(String args[]) {
14)         System.out.println(A.getRef());
15)         System.out.println(A.getRef());
16)         System.out.println(A.getRef());
17)     }
18) }

```

OUTPUT:

A@a6eb38a
A@a6eb38a
A@a6eb38a

Alternative Logic for Singleton Class:

```

1) class A {
2)     static A a = new A();
3)     private A() {
4)     }
5)     static A getRef() {
6)         return a;
7)     }
8) }
9) class Test {
10)     public static void main(String args[]) {
11)         System.out.println(A.getRef());
12)         System.out.println(A.getRef());
13)         System.out.println(A.getRef());
14)     }
15) }

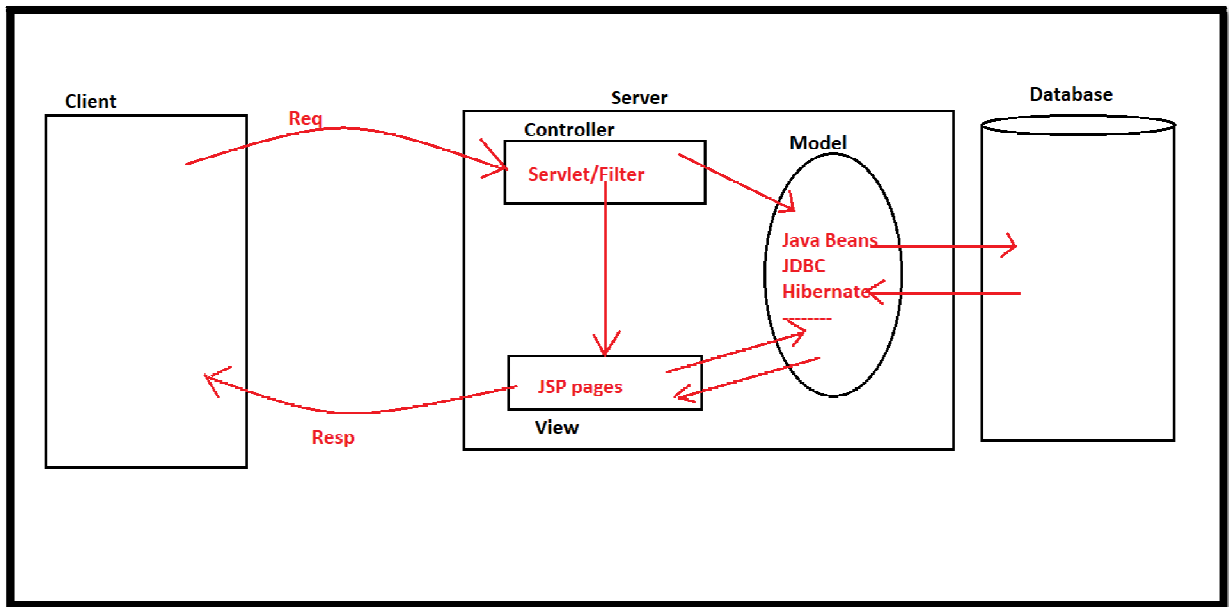
```

OUTPUT:

A@69cd2e5f
A@69cd2e5f
A@69cd2e5f

MVC is a design pattern, it will provide a standard structure to prepare web applications/GUI applications, where in MVC based applications we must use a servlet/ filter [A Java class] as controller, a set of JSP pages/Html pages as view part and a java bean or EJB component or JDBC program or Hibernate program as Model component. As per MVC Arch rules and regulations, only one controller must be provided for application.

that is, the class which we used as controller must provide one object, therefore, controller class must be singleton class.



EX1: Struts is MVC based Framework, where "ActionServlet" is controller, so it is Singleton class.

EX2: JSF[Java Server Faces] is MVC based framework, where "FacesServlet" is controller, so it is Singleton class.

Class.forName(--)

Consider the following program

```
1) class A
2) {
3)     static
4)     {
5)         System.out.println("Class Loading");
6)     }
7)     A()
8)     {
9)         System.out.println("Object Creating");
10)    }
11) }
12) class Test
13) {
14)     public static void main(String[] args) throws Exception
15)     {
16)         A a=new A();
```

```
17) }  
18)}
```

When we access a constructor along with "new" keyword then JVM will perform the following actions automatically.

1. JVM will load the respective class bytecode to the memory.
2. JVM will create object for the loaded class.

As per the requirement, if we want to load class byte code to the memory with out creating object then we have to use the following method from java.lang.Class class.

```
public static Class.forName(String class_Name) throws ClassNotFoundException
```

EX: Class c = Class.forName("A");

When JVM encounter the above instruction, JVM will perform the following actions.

- 1) JVM will take class name from forName(-) method.
- 2) JVM will search for its .class file at current location, at java predefined library and at the locations referred by "classpath" environment variable.
- 3) If the required .class file is not available at all the above locations then JVM will rise an exception like "java.lang.ClassNotFoundException".
- 4) If the required .class file is available at either of the above locations then JVM will load its bytecode to the memory.
- 5) After loading class bytecode to the memory, JVM will take metadata of the loaded class like class name, super class details, implemented interfaces details, variables details, methods details,.... and JVM will store them by creating java.lang.Class object in heap memory and JVM will return the generated Class object reference value from Class.forName(-);

EX:

```
1) class A  
2) {  
3)     static  
4)     {  
5)         System.out.println("Class Loading");  
6)     }  
7)     A()  
8)     {  
9)         System.out.println("Object Creating");  
10)    }  
11) }  
12) class Test  
13) {
```



```

14) public static void main(String[] args)throws Exception
15) {
16)     Class c=Class.forName("A");
17) }
18) }

```

OUTPUT: Class Loading

After loading class bytecode by using Class.forName(-) method, if we want to create object explicitly then we have to use the following method.

`public Object newInstance()throws InstantiationException, IllegalAccessException`

EX: Object obj=c.newInstance();

When JVM encounter the above instruction, JVM will perform the following actions.

- 1) JVM will goto the loaded class bytecode and JVM will check whether 0-arg and non-private constructor is existed or not.
- 2) If 0-arg and non-private constructor is existed then JVM will execute that constructor and JVM will create object for the loaded class.
- 3) If the constructor parameterized constructor without 0-arg constructor then JVM will rise an excewption like "java.lang.InstantiationException".
- 4) If the constructor is private constrictor then JVM will rise an exception like "java.lang.IllegalAccessException".

Note: If the constructor is both parameterized and 0-arg then JVM will rise "java.lang.InstaitiationException" only.

EX:

```

1) class A
2) {
3)     static
4)     {
5)         System.out.println("Class Loading");
6)     }
7)     A()
8)     {
9)         System.out.println("Object Creating");
10)    }
11) }
12) class Test
13) {
14)     public static void main(String[] args)throws Exception
15)     {

```

```
16)    Class c=Class.forName("A");  
17)    Object obj=c.newInstance();  
18)    }  
19) }
```

OUTPUT:

Class Loading

Object Creating

In JDBC Appl, we are going to use Driver to map JAVA representations to Database representations and Database representations to Java representations. In Jdbc applications, we must load driver class, not to create object for Driver class, for this, we have to use "Class.forName(-)"

In general, all server side components like Servlets, JSPs, EJBs,... are executed by the server[Container] by following their lifecycle actions like loading, instantiation, innitialization,..... In this context, to perform server side components loading Container will use "Class.forName(-)" method and to perform instantiation lifecycle action container will use "newInstance()" method internally.

Final Keyword:

final is a Java Keyword it can be used to declare constant expressions.

In java applications, 'final' keyword is able to improve security.

In Java applications, there are three ways to utilize 'final' keyword.

- 1) final variable
- 2) final method
- 3) final class

1) final Variable:

final variable is a variable, it will not allow modifications on its value.

EX:

```
final int i=10;
```

```
i=i+10;----> Compilation Error
```

EX:

```
for(final int i=0;i<10;i++) ----> Compilation Error
```

```
{  
    System.out.println(i);
```

```
}
```

NOTE: In general, in bank applications, after creating an account it is possible to change the account details like account name, address details....but it is not possible to update 'accNo' value once it is created. Due to this reason, we have to declare 'accNo' variable as 'final' variable.

2) final Method:

- final method is a Java method, it will not allow method overriding.
- In method overriding, we have to provide same method with different implementation at both super class and at sub class, where super class method never be declared as final irrespective of sub class method final.

EX1:

```
1) class A {
2)     final void m1() {
3)         System.out.println("m1-A");
4)     }
5) }
6) class B extends A {
7)     void m1() {
8)         System.out.println("m1-B");
9)     }
10) }
11) class Test {
12)     public static void main(String[] args) {
13)         A a = new B();
14)         a.m1();
15)     }
16) }
```

Status: Compilation Error.

EX2:

```
1) class A {
2)     final void m1() {
3)         System.out.println("m1-A");
4)     }
5) }
6) class B extends A {
7)     final void m1() {
8)         System.out.println("m1-B");
9)     }
10) }
```

```

11) class Test {
12)     public static void main(String[] args) {
13)         A a = new B();
14)         a.m1();
15)     }
16) }

```

Status: Compilation Error

EX3:

```

1) class A {
2)     void m1() {
3)         System.out.println("m1-A");
4)     }
5) }
6) class B extends A {
7)     final void m1() {
8)         System.out.println("m1-B");
9)     }
10) }
11) class Test {
12)     public static void main(String[] args) {
13)         A a = new B();
14)         a.m1();
15)     }
16) }

```

Status: No Compilation Error.

3) final Class:

- Final class is a Java class, it will not allow inheritance.
- In Java applications, super classes never be declared as final classes, but sub classes may be final.

EX:

```

1) final class A
2) {
3) }
4) class B extends A
5) {
6) }

```

Status:Invalid

EX:

```
1) final class A
2) {
3) }
4) final class B extends A
5) {
6) }
```

Status:Invalid

EX:

```
1) class A
2) {
3) }
4) final class B extends A
5) {
6) }
```

Status:Valid

In Java applications to declare constant variables Java has provided a convention like to declare constants with "public static final".

EX:

In System Class:

```
public static final PrintStream out;
public static final InputStream in;
public static final PrintStream err;
```

In Thread Class:

```
public static final int MIN_PRIORITY=1;
public static final int NORM_PRIORITY=5;
public static final int MAX_PRIORITY=10;
```

EX:

```
1) class User_Status {
2)     public static final String AVAILABLE="Available";
3)     public static final String BUSY="Busy";
4)     public static final String IDLE="Idle";
5) }
6) class Test {
7)     public static void main(String args[]) {
8)         System.out.println(User_Status.AVAILABLE);
```

```

9)      System.out.println(User_Status.BUSY);
10)     System.out.println(User_Status.IDLE);
11)    }
12) }

```

OUTPUT:

Available

Busy

Idle

To declare constant variables in Java applications if we use the above convention then we are able to get the following problems.

- 1) We must declare "public static final" for each and every constant variable explicitly.
- 2) It is possible to allow multiple data types to represent one type, it will reduce typedness in Java applications.
- 3) If we access constant variables then these variables will display their values, here constant variable values may or may not reflect the actual meaning of constant variables.

To overcome all the problems, we have to go for "enum".

In case of enum,

- 1) All the constant variables are by default "public static final", no need to declare explicitly.
- 2) All the constant variables are by default the same enum type, it will improve typedness in Java applications.
- 3) All the constant variables are by default "Named Constants" that is, these constant variables are displaying their names instead of their values.

Syntax:

```

[Access_modifier] enum Enum_Name
{
    ----- List of constants-----
}

```

EX:

```

1) enum User_Status {
2)     AVAILABLE,BUSY,IDLE;
3) }
4) class Test {

```

```

5)      public static void main(String args[]) {
6)          System.out.println(User_Status.AVAILABLE);
7)          System.out.println(User_Status.BUSY);
8)          System.out.println(User_Status.IDLE);
9)      }
10) }

```

OUTPUT:

Available
 Busy
 Idle

NOTE: The default super class for every enum is "java.lang.Enum" class and "Object" class is Super class to "Enum" class.

If we compile the above Java file then Compiler will translate "User_Status" enum into "User_Status" final class like below.

```

final class User_Status extends java.lang.Enum {
    public static final MailStatus AVAILABLE;
    public static final MailStatus BUSY;
    public static final MailStatus IDLE;
    -----
}

```

NOTE: In Java, java.lang.Object class is common and default super class for all the classes. Similarly, All the Java enums are having a common and default super class that is "java.lang.Enum".

NOTE: In Java applications, it is possible to implement inheritance between two classes but it is not possible to implement inheritance between two "enums", because, by default, enums are final classes.

In Java applications, we can utilize enum like as classes, where we can provide normal variables, methods, constructors....

EX

```

1)  enum Apple {
2)      A(500),B(250),C(100);
3)      int price;
4)      Apple(int price) {
5)          this.price = price;
6)      }
7)      public int getPrice() {
8)          return price;

```

String Manipulations

In C and C++ applications, to perform String operations, C and C++ programming languages have provided some predefined library in the form of the functions.

In Java, to perform String operations JAVA has provided the following predefined classes.

- `java.lang.String`
- `java.lang.StringBuffer`
- `java.lang.StringBuilder`
- `java.util.StringTokenizer`

Q) What is the difference between *String* and *StringBuffer*?

- String class objects are immutable objects, where Immutable objects are not allowing modifications over their content, if we are trying to perform modifications over immutable object content then operations are allowed, but, the resultant data will not be stored back in original object, where the resultant data will be stored by creating new object.
- StringBuffer objects are mutable objects, they are able to allow modifications on their content.

Q) What are the differences between *StringBuffer* and *StringBuilder*?

- StringBuffer class was introduced in JDK1.0 version.
StringBuilder class was introduced in JDK5.0 version.
- StringBuffer is synchronized.
StringBuilder is not synchronized.
- Almost all the methods are synchronized in StringBuffer.
No method is synchronized in StringBuilder.
- StringBuffer is able to allow only one thread to access data.
StringBuilder is able to allow more than one thread to access data.
- StringBuffer is following sequential execution.
StringBuilder is following parallel execution.
- StringBuffer will increase execution time.
StringBuilder will reduce execution time.
- StringBuffer will reduce application performance.
StringBuilder will increase application performance.

- **StringBuffer** is able to give guarantee for Data Consistency.
StringBuilder is unable to give guarantee for data consistency.

- **StringBuffer** is threadsafe.
StringBuilder is not thread safe.

Q) What is String tokenization and how it is possible to perform String Tokenization in Java Applications?

- The process of dividing a string into no of tokens is called as String Tokenization.
- To perform String Tokenization JAVA has provided a predefined class in the form of `java.util.StringTokenizer`.

To perform String Tokenization in java applications we have to use the following steps.

- **Create StringTokenizer Class Object:**

To create StringTokenizer class object we have to use the following constructor.

```
public StringTokenizer(String data)
```

EX: `StringTokenizer st=new StringTokenizer("Durga Software Solutions");`

When JVM encounter the above instruction, JVM will take the provided String, JVM will divide the provided String into no of tokens and JVM will store the generated tokens by creating object for StringTokenizer class.

NOTE: When StringTokenizer class object is created with the no of tokens then a pointer or cursor will be created before the first token.

To get no of tokens which are existed in StringTokenizer class object we have to use the following method.

```
public int countTokens()
```

- **Retrieve Tokens from StringTokenizer Object:**

To retrieve tokens from StringTokenizer object we have to use the following steps for each and every token.

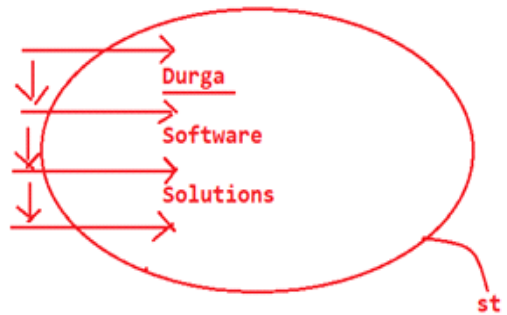
- Check whether more tokens are available or not from the current cursor position by using the following method. `public boolean hasMoreTokens()`
- It will return true value if at least next token is existed.
- It will return false value if no more tokens are existed.
- If atleast next token is available then read next token and move cursor to next position by using the following method.
`public String nextToken()`

EX:

```
1) import java.util.*;
2) class Test
3) {
4)     public static void main(String[] args)
5)     {
6)         StringTokenizer st=new StringTokenizer("Durga Software Solutions");
7)         System.out.println("No Of Tokens:"+st.countTokens());
8)         while(st.hasMoreTokens())
9)         {
10)            System.out.println(st.nextToken());
11)        }
12)    }
13)}
```

```
StringTokenizer st = new StringTokenizer("Durga Software Solutions");
int count = st.countTokens()

while(st.hasMoreTokens())
{
    Sopln(st.nextToken());
}
```



String Class Library:

Constructor:

- public String()

It able to create an empty String object.

EX: String str = new Stirng();

- public String(String str)

This constructor can be used to create String class object with the specified data.

EX: String str = new String("Durga Software Solutions");
System.out.println(str);

Output: Durga Software Solutions

Q) What is the difference between the following 2 Statements

a) `String str = new String("Durga Software Solutions");`

b) `String str = "Durga Software Solutions";`

- To create String class object if we use first statement then one String object will be created at heap memory as per "new" keyword and another String object will be created at String Constant Pool Area inside method Area as per "".
- To create String object if we use second statement then String object will be created at String Constant Pool Area only in Method Area.

Note: The objects which are created in String Constant Pool Area are not eligible for Garbage Collection. The objects which are created in Heap memory are eligible for Garbage Collection.

3) public String(byte[] b)

This constructor can be used to create String object with the String equivalent of the specified byte[].

EX: `byte[] b = {65, 66, 67, 68, 69, 70};`

`String str = new String(b);`

`System.out.println(str);`

Output: ABCDEF

4) public String(byte[] b, int start_index, int no_Of_Chars)

This constructor can be used to create String class object with the String equivalent of the specified byte[] starts from the specified start index and up to the specified no of elements.

EX: `byte[] b = {65, 66, 67, 68, 69, 70};`

`String str = new String(b, 2, 3);`

`System.out.println(str);`

Output: CDE

5) public String(char[] ch)

This constructor can be used to create String object with the String equivalent of the specified char[].

EX: `char[] ch = {'A', 'B', 'C', 'D', 'E', 'F'};`

`String str = new String(ch);`

`System.out.println(str);`

Output: ABCDEF

6) public String(char[] ch, int start_Index,int no_Of_Chars)

It will create String object with the String equivalent of the specified char[] starts from the specified start index and up to the specified no of element.

EX: char[] ch = {'A','B','C','D','E','F'};
String str = new String(ch,2,3);
System.out.println(str);
Output: CDE

Note: Constructors 3 and 4 are used to convert data from byte[] to String and constructors 5 and 6 are used to convert data from char[] to String.

Methods:

- **public int length()**

This method will return an integer value representing the no of characters existed in the String including spaces.

EX: String str = new String("Durga Software Solutions");
System.out.println(str.length());
Output: 24

- **public String concat(String str)**

This method will add the specified String to String object content in immutable manner.

EX:
String str1 = new String("Durga ");
String str2 = str1.concat("Software ");
String str3 = str2.concat("Solutions");
System.out.println(str1);
System.out.println(str2);
System.out.println(str3);

Output: Durga
Durga Software
Durga Software Solutions

EX:
String str = "Durga".concat("Software ").concat("Solutions");
System.out.println(str);

Output: Durga Software Solutions

- **public boolean equals(Object obj)**

This method will check whether the two String objects content is same or not. if two string objects content is same then equals(-) method will return "true" value otherwise equals(-) method will return "false" value.

Q) What is the difference between == Operator and equals(-) Method?

- '=' is basically a comparison operator, it will check whether the provided operand values are same or not, where operands may be normal primitive variables or object reference variables.
- Initially equals(-) method was defined in java.lang.Object class , it was implemented in such a way that to provide comparison between two object reference values.
- String class has overridden Object class equals(-) method in such a way that to provide comparison between two String object contents

EX:

```
1) class A
2) {
3) }
4) class Test
5) {
6)     public static void main(String[] args)
7)     {
8)         A a1=new A();
9)         A a2=new A();
10)
11)         int i=10;
12)         int j=10;
13)
14)         String str1=new String("abc");
15)         String str2=new String("abc");
16)
17)         System.out.println(i == j); // true
18)         System.out.println(a1 == a2); // false
19)         System.out.println(str1 == str2); // false
20)         System.out.println(a1.equals(a2)); // false
21)         System.out.println(str1.equals(str2)); //true
22)     }
23) }
```

public boolean equalsIgnoreCase(String str)

In String class, equals(-) method will perform case sensitive comparison between two String object contents, but, equalsIgnoreCase(-) method will perform case insensitive comparison between two String objects content.

EX:

```
String str1 = new String("abc");  
String str2 = new String("ABC");  
System.out.println(str1.equals(str2));  
System.out.println(str1.equalsIgnoreCase(str2));
```

Output:

false
true

- **public int compareTo(String str)**

This method will check whether two string objects contents are existed in dictionary order or not.

str1.compareTo(str2)

- a) If str1 come first when compared with str2 in dictionary order then compareTo() method will return -ve value.
- b) If str2 come first when compared with str1 in dictionary order then compareTo() method will return +ve value.
- c) If str1 and str2 are available at the same position in dictionary order then compareTo() method will return 0 value.

EX:

```
String str1 = new String("abc");  
String str2 = new String("def");  
String str3 = new String("abc");  
System.out.println(str1.compareTo(str2));  
System.out.println(str2.compareTo(str3));  
System.out.println(str3.compareTo(str1));
```

Output: -3
3
0

- **public boolean startsWith(String str)**

It will check whether the String starts with the specified String or not.

- **public boolean endsWith(String str)**

It will check whether the String ends with the specified String or not.

- **public boolean contains(String str)**

It will check whether String contains the specified String or not.

EX:

```
String str=new String("Durga Software Solutions");  
System.out.println(str.startsWith("Durga"));  
System.out.println(str.endsWith("Solutions"));  
System.out.println(str.contains("Software"));
```

OUTPUT:

```
true  
true  
true
```

- **public String replace(char old_Char, char new_Char)**

This method will replace the specified old char with the specified new char.

- **public char charAt(int index)**

It will return a character which is existed at the specified index value.

- **public int indexOf(String str)**

It will return an index value where the first occurrence of the specified String.

- **public int lastIndexOf(String str)**

It will return an index value where the last occurrence of the specified String.

EX:

```
String str=new String("Durga Software Solutions");  
System.out.println(str);  
System.out.println(str.replace('S', 's'));  
System.out.println(str.charAt(6));  
System.out.println(str.indexOf("So"));  
System.out.println(str.lastIndexOf("So"));
```

Output: Durga Software Solutions

Durga software solutions

S

6

15

- **public String substring(int start_index)**

This method return a substring starts from the specified index value.

- **public String substring (int start_Index, int end_Index)**

This method will return a substring starts from the specified start index and up to the specified end index.

EX:

```
String str = new String("Durga Software Solutions");
```

```
System.out.println(str.substring(6));
```

```
System.out.println(str.substring(6,14));
```

Output:

Software Solutions

Software

- **public byte[] getBytes()**

This method will convert data from String to byte[].

- **public char[] toCharArray()**

This method will convert data from String to char[].

EX:

```
1) String str = new String("Durga Software Solutions");
```

```
2) byte[] b = str.getBytes();
```

```
3) for(int i=0; i<b.length; i++)
```

```
4) {
```

```
5)     System.out.print(b[i]+" ");
```

```
6) }
```

```
7) System.out.println();
```

```
8) char[] ch = str.toCharArray();
```

```
9) for(int i=0; i<ch.length; i++)
```

```
10){
```

```
11)     System.out.print(ch[i]+" ");
```

```
12)}
```


public String[] split(String str)

It will divide the String into no pieces on the basis of the provided String.

EX:

```
String str = new String("Durga Software Solutions");
String[] s = str.split(" ");
for(int i=0; i<s.length; i++)
{
    System.out.println(s[i]);
}
```

- public String trim()

This method will remove pre-spaces and post-spaces of a particular String.

- public String toLowerCase()

It will convert String into lower case letters.

- public String toUpperCase()

It will convert String into Upper case letters.

EX: String str = new String(" Durga Software Solutions ");
System.out.println(str);
System.out.println(str.trim());
String str1 = new String("Durga Software Solutions");
System.out.println(str1.toLowerCase());
System.out.println(str1.toUpperCase());

StringBuffer:

Costructors:

- public StringBuffer()

This constructor can be used to create an empty StringBuffer object with 16 elements as initial capacity.

EX: StringBuffer sb=new StringBuffer();
System.out.println(sb.capacity());

Output: 16

- **public StringBuffer(int capacity)**

This constructor can be used to create an empty StringBuffer object with the specified capacity value.

EX: `StringBuffer sb=new StringBuffer(10);`
`System.out.println(sb.capacity());`

Output: 10

- **public StringBuffer(String str)**

This constructor can be used to create StringBuffer object with the specified data.

$\text{New_capacity} = \text{Initial_Capacity} + \text{data_Length};$

EX: `StringBuffer sb = new StringBuffer("abc");`
`System.out.println(sb);`
`System.out.println(sb.capacity());`

Output: abc
19

Methods:

- **public StringBuffer append(String data)**

This method will append the specified data to the StringBuffer object content.

EX: `StringBuffer sb1 = new StringBuffer("Durga ");`
`StringBuffer sb2 = sb1.append("Software ");`
`StringBuffer sb3 = sb2.append("Solutions");`
`System.out.println(sb1);`
`System.out.println(sb2);`
`System.out.println(sb3);`

Output: Durga Software Solutions
Durga Software Solutions
Durga Software Solutions

- **public void ensureCapacity(int capacity)**

This method can be used to set a particular capacity value to StringBuffer object. If the provided capacity value is less than 16 then StringBuffer object will take 16 as capacity value. If the provided capacity value is greater than 16 and less than 34 then StringBuffer will take 34 as capacity value. If the provided capacity value is greater than 34 then StringBuffer object will take the specified value as capacity value.

EX: `StringBuffer sb = new StringBuffer();
sb.ensureCapacity(10);
System.out.println(sb.capacity());`

Output: 16

- **public StringBuffer reverse()**

This method will reverse the content of StringBuffer object.

EX: `StringBuffer sb = new StringBuffer("Durga Software Solutions");
System.out.println(sb);
System.out.println(sb.reverse());`

public StringBuffer delete(int start index, int end index)

This method can be used to delete a string from StringBuffer object content starts from the specified start index and up to the specified end index.

EX:

`StringBuffer sb = new StringBuffer("Durga Software Solutions");
System.out.println(sb);
System.out.println(sb.delete(6,14));`

Output:

Durga Software Solutions
Durga Solutions

- **public StringBuffer insert(int index, String str)**

This method can be used to insert the specified String at the specified index in StringBuffer object.

EX: `StringBuffer sb = new StringBuffer("Durga Solutions");
System.out.println(sb);
System.out.println(sb.insert(6,"Software"));`

Output:

Durga Solutions
Durga Software Solutions

Exception Handling